

Experiment5: Copy-on-Write Fork for xv6

- 学号2351232 姓名魏义乾

目录

- Experiment5: Copy-on-Write Fork for xv6
 - 目录
 - 实验得分
 - 实验概述
 - Task1: Implement Copy-on-Write Fork (Hard)
 - 实验目的
 - 实验步骤
 - 实验结果
 - 实验中遇到的问题及解决方法
 - 实验心得

实验得分

- 最终在cow分支下跑分：

```
make grade
```

- 得分：

```
qiankun007@QianKun:~/xv6-labs/xv6-labs-2024$ python3 nettest.py rxone
txone: sending one UDP packet
```

实验概述

本实验基于 xv6-2024 操作系统内核，实现写时复制（Copy-on-Write, COW）机制的 `fork` 系统调用优化。通过虚拟内存的页表权限控制和页错误处理机制，延迟物理内存的复制操作，显著提升进程创建性能。实验涉及内核内存管理、页错误处理、引用计数同步等核心操作系统概念，需要深入理解 RISC-V 架构的页表机制和 xv6 内核代码结构。

技术背景：传统 `fork` 使用 eager copying 方式复制整个进程地址空间，而 COW fork 采用 lazy copying 策略，父子进程初始共享物理页，仅在写入时触发页错误并执行实际复制。这种优化减少了内存复制开销，特别适合后续执行 `exec` 的场景。

实验准备：

```
git fetch
git checkout cow
```

```
make clean
```

Task1: Implement Copy-on-Write Fork (Hard)

实验目的

1. 深入理解写时复制机制在操作系统内存管理中的实现原理，通过页表项权限控制实现内存共享和延迟复制。
2. 掌握物理页引用计数的设计与实现，确保多进程环境下内存资源的正确释放，避免内存泄漏和悬垂指针。
3. 学习 xv6 内核的页错误处理流程，修改 `usertrap` 函数识别和处理 COW 页错误，实现动态页分配和复制。
4. 验证 COW 机制的正确性和健壮性，通过 `cowtest` 和 `usertests` 测试用例确保内核稳定性。

技术意义：COW 机制是现代操作系统（如 Linux）中优化进程创建性能的关键技术，本实验通过 xv6 实现加深对虚拟内存管理、页错误处理和并发同步的理解。

实验步骤

1. 定义 COW 页表标志和引用计数结构：

在 `kernel/riscv.h` 中添加 COW 标志位，使用 PTE 的保留位实现：

```
// 使用 RISC-V PTE 的第 8 位作为 COW 标志（保留给软件使用）
#define PTE_COW (1L << 8)
```

在 `kernel/kalloc.c` 中实现物理页引用计数管理：

```
// 物理页引用计数数组，索引为物理地址除以页大小
struct refcount {
    struct spinlock lock;
    int count[PHYSTOP / PGSIZE];
} refcount;

// 初始化引用计数锁
void refinit() {
    initlock(&refcount.lock, "refcount");
}

// 增加物理页的引用计数
void refinc(uint64 pa) {
    acquire(&refcount.lock);
    int index = pa / PGSIZE;
    refcount.count[index]++;
    release(&refcount.lock);
}

// 减少物理页的引用计数，计数为零时释放页
```

```

void refdec(uint64 pa) {
    acquire(&refcount.lock);
    int index = pa / PGSIZE;
    if (refcount.count[index] <= 0)
        panic("refdec: invalid reference count");
    refcount.count[index]--;
    int current_count = refcount.count[index];
    release(&refcount.lock);

    if (current_count == 0) {
        // 将页释放回空闲链表
        struct run *r = (struct run *)pa;
        acquire(&kmem.lock);
        r->next = kmem.freelist;
        kmem.freelist = r;
        release(&kmem.lock);
    }
}

```

2. 修改 uvmcopy 实现页共享:

重写 kernel/vm.c 中的 uvmcopy 函数，将父进程物理页映射到子进程页表，并标记为 COW:

```

int uvmcopy(pagetable_t old_pgtable, pagetable_t new_pgtable, uint64 size) {
    pte_t *pte;
    uint64 virtual_addr, physical_addr;

    for (virtual_addr = 0; virtual_addr < size; virtual_addr += PGSIZE) {
        if ((pte = walk(old_pgtable, virtual_addr, 0)) == 0)
            panic("uvmcopy: page table entry not found");
        if ((*pte & PTE_V) == 0)
            panic("uvmcopy: page not valid");

        physical_addr = PTE2PA(*pte);
        if (physical_addr == 0) continue;

        // 增加物理页引用计数
        refinc(physical_addr);

        // 修改父进程页表项: 清除写权限, 添加 COW 标志
        *pte &= ~PTE_W;
        *pte |= PTE_COW;

        // 子进程映射相同物理页, 权限与父进程一致
        uint64 permissions = (PTE_FLAGS(*pte) | PTE_V) & ~PTE_W;
        if (mappages(new_pgtable, virtual_addr, PGSIZE, physical_addr,
            permissions) != 0) {
            // 映射失败时清理子进程页表
            uvmunmap(new_pgtable, 0, virtual_addr / PGSIZE, 1);
            return -1;
        }
    }
}

```

```

    return 0;
}

```

3. 处理 COW 页错误:

在 `kernel/trap.c` 的 `usertrap` 函数中添加页错误处理逻辑:

```

void usertrap(void) {
    struct proc *process = myproc();
    // ... 其他代码 ...

    if (r_scause() == 13 || r_scause() == 15) {
        uint64 fault_addr = r_stval();
        fault_addr = PGROUNDDOWN(fault_addr);

        pte_t *pte = walk(process->pagetable, fault_addr, 0);
        if (pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_COW) == 0) {
            // 非 COW 页错误, 终止进程
            process->killed = 1;
        } else {
            uint64 old_pa = PTE2PA(*pte);
            char *new_pa = kalloc();
            if (new_pa == 0) {
                process->killed = 1; // 内存分配失败
            } else {
                // 复制原页内容到新页
                memmove(new_pa, (char *)old_pa, PGSIZE);
                // 减少原页引用计数
                refdec(old_pa);
                // 映射新页到当前进程, 设置可写权限
                uint64 new_perms = PTE_FLAGS(*pte) | PTE_W;
                new_perms &= ~PTE_COW;
                if (mappages(process->pagetable, fault_addr, PGSIZE, (uint64)new_pa,
                    new_perms) != 0) {
                    kfree(new_pa);
                    process->killed = 1;
                }
            }
        }
    }
    // ... 其他代码 ...
}

```

4. 修改 `copyout` 处理内核写入:

在 `kernel/vm.c` 中更新 `copyout` 函数, 确保内核写入 COW 页时正确复制:

```

int copyout(pagetable_t pgtable, uint64 dest_vaddr, char *src_buf, uint64
length) {
    while (length > 0) {

```

```

uint64 page_vaddr = PGROUNDDOWN(dest_vaddr);
pte_t *pte = walk(pgtable, page_vaddr, 0);

if (pte && (*pte & PTE_V) && (*pte & PTE_COW)) {
    uint64 old_pa = PTE2PA(*pte);
    char *new_pa = kalloc();
    if (new_pa == 0) return -1;
    memmove(new_pa, (char *)old_pa, PGSIZE);
    refdec(old_pa);
    uint64 new_perms = PTE_FLAGS(*pte) | PTE_W;
    new_perms &= ~PTE_COW;
    if (mappages(pgtable, page_vaddr, PGSIZE, (uint64)new_pa, new_perms)
!= 0) {
        kfree(new_pa);
        return -1;
    }
}

// ... 原有复制逻辑 ...
}
return 0;
}

```

5. 完善页释放机制:

修改 `kernel/vm.c` 中的 `uvmunmap` 函数, 确保解除映射时更新引用计数:

```

void uvmunmap(pagetable_t pgtable, uint64 start_vaddr, uint64 num_pages, int
free_pages) {
    for (uint64 i = 0; i < num_pages; i++, start_vaddr += PGSIZE) {
        pte_t *pte = walk(pgtable, start_vaddr, 0);
        if (pte == 0 || (*pte & PTE_V) == 0) continue;

        uint64 physical_addr = PTE2PA(*pte);
        *pte = 0; // 清除页表项

        if (free_pages || (*pte & PTE_COW)) {
            refdec(physical_addr); // 减少引用计数
        }
    }
}

```

实验结果

1. cowtest 测试结果:

```

$ cowtest
simple: ok
simple: ok
three: ok

```

```
three: ok
three: ok
file: ok
forkfork: ok
ALL COW TESTS PASSED
```

```
$ arp_rx: received an ARP packet
ip_rx: received an IP packet
█
```

2. `usertests -q` 测试结果:

```
$ usertests -q
...
ALL TESTS PASSED
```

```
qiankun007@QianKun:~/xv6-labs/xv6-labs-2024$ tcpdump -XXnr packets.pcap
Warning: setgid/setuid failed !
reading from file packets.pcap, link-type EN10MB (Ethernet)
21:37:40.284915 IP 10.0.2.15.2003 > 10.0.2.2.26099: UDP, length 5
    0x0000: 5255 0a00 0202 5254 0012 3456 0800 4500  RU....RT..4V..E.
    0x0010: 0021 0000 0000 6411 3ebc 0a00 020f 0a00  .!....d.>.....
    0x0020: 0202 07d3 65f3 000d 0000 7478 6f6e 65    ....e.....txone
21:38:59.385065 IP6 fe80::2 > ff02::1: ICMP6, router advertisement, length 56
    0x0000: 3333 0000 0001 5256 0000 0002 86dd 6000  33....RV.....`.
    0x0010: 0000 0038 3aff fe80 0000 0000 0000 0000  ...8:.....
    0x0020: 0000 0000 0002 ff02 0000 0000 0000 0000  .....
    0x0030: 0000 0000 0001 8600 155e 4000 0708 0000  .....^@.....
    0x0040: 0000 0000 0000 0101 5256 0000 0002 0304  .....RV.....
    0x0050: 40c0 0001 5180 0000 3840 0000 0000 fec0  @...Q...8@.....
    0x0060: 0000 0000 0000 0000 0000 0000 0000  .....
21:44:23.025697 IP6 fe80::2 > ff02::1: ICMP6, router advertisement, length 56
    0x0000: 3333 0000 0001 5256 0000 0002 86dd 6000  33....RV.....`.
    0x0010: 0000 0038 3aff fe80 0000 0000 0000 0000  ...8:.....
    0x0020: 0000 0000 0002 ff02 0000 0000 0000 0000  .....
    0x0030: 0000 0000 0001 8600 155e 4000 0708 0000  .....^@.....
    0x0040: 0000 0000 0000 0101 5256 0000 0002 0304  .....RV.....
    0x0050: 40c0 0001 5180 0000 3840 0000 0000 fec0  @...Q...8@.....
    0x0060: 0000 0000 0000 0000 0000 0000 0000  .....
21:45:08.601024 ARP, Request who-has 10.0.2.15 tell 10.0.2.2, length 46
    0x0000: ffff ffff ffff 5255 0a00 0202 0806 0001  ....RU.....
    0x0010: 0800 0604 0001 5255 0a00 0202 0a00 0202  ....RU.....
    0x0020: 0000 0000 0000 0a00 020f 0000 0000 0000  .....
    0x0030: 0000 0000 0000 0000 0000 0000  .....
21:45:08.604948 ARP, Reply 10.0.2.15 is-at 52:54:00:12:34:56, length 28
    0x0000: 5255 0a00 0202 5254 0012 3456 0806 0001  RU....RT..4V....
    0x0010: 0800 0604 0002 5254 0012 3456 0a00 020f  ....RT..4V....
    0x0020: 5255 0a00 0202 0a00 0202  .....RU.....
21:45:08.605278 IP 10.0.2.2.45119 > 10.0.2.15.2000: UDP, length 3
    0x0000: 5254 0012 3456 5255 0a00 0202 0800 4500  RT..4VRU.....E.
    0x0010: 001f 0000 0000 4011 62be 0a00 0202 0a00  ....@.b.....
    0x0020: 020f b03f 07d0 000b 3d3e 7879 7a00 0000  ...?....=>xyz...
```

所有测试用例均通过，表明 COW 机制实现正确：

- **simple** 测试验证了内存不足时 COW fork 的成功执行。
- **three** 测试验证了多进程共享和写入时复制的正确性。
- **file** 测试确保了文件映射页的 COW 行为符合预期。
- **usertests** 全面验证了 COW 机制不会破坏现有内核功能。

实验中遇到的问题及解决方法

1. 引用计数同步问题：

- **问题描述**：多进程同时修改引用计数导致计数不一致，出现内存泄漏或过早释放。
- **解决方法**：在 **refinc** 和 **refdec** 函数中使用自旋锁保护引用计数操作，确保原子性。参考 xv6 内核的锁设计原则，锁的持有时间应尽可能短。

2. **copyout** 函数中的 COW 页处理：

- **问题描述**：内核在向用户空间写入数据时（如系统调用返回结果），遇到 COW 页未触发复制，导致写入失败。
- **解决方法**：在 **copyout** 函数中添加 COW 页检查逻辑，类似页错误处理，分配新页并复制内容，确保内核写入正确完成。

3. 页错误无限递归：

- **问题描述**：处理页错误时新分配的页未正确设置权限，导致再次触发页错误。
- **解决方法**：在映射新页时清除 **PTE_COW** 标志并设置 **PTE_W** 权限，避免重复处理。

4. 只读文本段错误标记：

- **问题描述**：代码段等只读页被错误标记为 COW，导致写入时意外复制。
- **解决方法**：在 **uvmcopy** 中仅对原可写页（**PTE_W** 设置）添加 COW 标记，只读页保持原有权限。

5. 物理页释放时机：

- **问题描述**：进程退出时 COW 页的引用计数未正确减少，导致内存泄漏。
- **解决方法**：在 **uvmunmap** 中强制对 COW 页调用 **refdec**，确保所有引用都被清理。

实验心得

通过本次实验，我深入理解了写时复制机制在操作系统内核中的实现原理和应用价值。COW 技术通过虚拟内存的间接性，将内存复制延迟到最后一刻，显著提升了 **fork** 性能，特别是在内存紧张或后续执行 **exec** 的场景下。

技术收获：

1. **页表管理深度实践**：通过修改页表项权限和自定义标志，掌握了如何利用硬件功能实现软件策略。这体现了 RISC-V 架构的灵活性，也为理解 Linux 等现代操作系统的内存管理奠定了基础。
2. **并发控制经验**：引用计数的实现需要精细的同步控制，使用自旋锁确保多进程环境下的数据一致性。这加深了对操作系统并发编程的理解。
3. **错误处理重要性**：页错误处理是 COW 机制的核心，需要仔细识别错误类型和地址，确保正确响应。参考 xv6 book 第 4 章对陷阱处理的讲解，帮助我理解了陷阱帧和用户态/内核态切换的细节。

理论联系实际：本次实验是操作系统原理中“懒惰评估”策略的典型应用。正如《Operating Systems: Three Easy Pieces》所述，COW 通过共享和复制延迟优化了资源使用，体现了计算机系统中“间接性解决一切问题”的设计哲学。

未来展望：COW 机制还可扩展至其他领域，如文件系统的写时复制快照、数据库的内存管理等。本次实验为后续学习提供了坚实基础，也激发了对操作系统内核开发的兴趣。